Neuromorphic Distributed Hash Table

An N-DHT explainer · v0.3.51 · 2026-05-16 · David A. Smith · YZ.social

> *The technology is shaped by the mission.*

The Mission

N-DHT is the engineering substrate for a privacy-first decentralized internet. The protocol is designed for a specific job: to carry the routing and publish/subscribe workload of that internet, on consumer devices, in real browsers, against the actual physics of the network it runs on. Everything else about the design — the architecture, the algorithms, the simulator, the red team, the deployment path, the application analysis — is shaped by that job.

The job is hard for an unfamiliar reason. The internet of 2026 has no shortage of clever distributed systems, but almost all of them are *distributed underneath, central on top*: a constellation of servers that the user reaches through a single corporate gateway. (The pattern is so common we have stopped noticing it. CDNs, federated identity providers, cloud auth, app stores, the "serverless" frameworks that run on three or four hyperscaler clouds — all examples of distribution-as-implementation-detail sitting under a single point of decision.) A real peer-to-peer internet — one where two strangers can find each other, exchange a message, and route around an outage without asking permission of any intermediary — requires a different kind of substrate. The substrate must be *federation-free at every layer*: addressing, routing, discovery, group communication, eventual delivery under churn.

A distributed hash table is the addressing layer of that substrate. Without one, every other peer-to-peer primitive collapses back into a centralized service: there is no way to find the user you want to talk to, no way to discover the file you want to download, no way to subscribe to the channel you want to follow, without somebody telling you where it lives. With one, those operations are mechanical: the network itself routes you to the right peer.

The mission is to build the addressing layer that a real peer-to-peer internet needs. The protocol described here is what that addressing layer looks like.

The Problem: Finding Stuff on a Network Without a Boss

Imagine you and a million strangers each hold one piece of a giant jigsaw puzzle. Someone walks up and asks for piece #438,291. How do you find it?

Option 1: Have a central directory. Some company keeps a giant list: "piece 438,291 is held by Alice." Easy to look up — but now that company controls everything. They can shut you down, spy on you, charge you money, or just go bankrupt and take the directory with them.

Option 2: Give every piece an "address," and design a system where the network *itself* knows how to route requests to the right person, with no central directory. This is what a **distributed hash table** (DHT) does.

DHTs are the machinery behind things you've probably used: BitTorrent, IPFS (a "decentralized" file storage system), parts of Ethereum, and the hidden services on Tor. Every node — every computer in the network — gets a random ID number. Every piece of content gets an ID too. To find content, you ask the node whose ID is "closest" to the content's ID.

The trick: how do you find that closest node when you don't know who's in the network and you only know a handful of peers?

The dominant DHT algorithm is called **Kademlia**, designed in 2002. It works like this:

Every node has a 160-bit ID — a very large random number. To measure "distance" between two IDs, you XOR them, comparing bit by bit. To find a target, you ask the closest peer you know. They tell you about peers *they* know that are even closer. You ask one of those. Repeat.

The math is clean: each step at least *halves* the remaining distance, so you reach any target in about $\log_2(N)$ hops. With a million nodes, that's 20 steps. Provably correct, used in production all over the internet.

There's just one problem.

Hops Are Cheap, But Time Is Expensive

20 hops sounds fast, but each hop sends a message between two random computers somewhere on Earth. If your peers are scattered randomly across the globe, the average pair sits about half the planet apart—roughly 100 milliseconds round trip.

20 hops × 100 ms = **2 seconds**. To find something. Every time.

For real-time applications — voice calls, online games, live messaging, push notifications — 2 seconds is unusable. The math says routing is logarithmic, which is fast; the physics says each step is slow because peers are far apart. That's the tension.

The protocol attacks the latency problem with two ideas, one of which is borrowed straight from neuroscience.

Idea #1: Put the Address in the Address

The first idea is almost embarrassingly simple. It's called **G-DHT** (Geographic DHT).

Kademlia node IDs are random. A node in Tokyo and a node in Berlin have IDs with no relationship to where they actually are. That's *why* hops are random and slow — random IDs mean random geography.

G-DHT changes one thing: the **first 8 bits** of every node ID encode where the node says it is on Earth.

G-DHT uses Google's S2 library, which divides Earth's surface into cells along a curve called a **Hilbert curve**. Its useful property: places near each other on Earth get cell numbers near each other. So XORing two S2 cell numbers gives a small result for nearby places and a large one for distant places.

Now your node ID looks like: `[8-bit geographic cell][56-bit hash of your public key]`.

The routing algorithm doesn't change at all, but XOR distance now approximately tracks physical distance. When a node looks for a "close" peer in ID space, it tends to find one that's also physically close. Local traffic stays local. The 20-hop world tour becomes a 13-hop journey around the neighborhood, with each hop maybe 7 ms instead of 100.

Total: about 91 ms instead of 2 seconds. **Roughly 20× faster** for regional traffic, just from changing the *structure* of the addresses.

The cost: nodes can lie about where they are. This is a "cooperative trust" assumption. Mitigations exist, but for now G-DHT trades some defense against location-spoofing for a large speedup.

Idea #2: A Network That Learns Like a Brain

The second idea — neuromorphic routing, which we call **N-DHT** — is the heart of the protocol. It asks a different question:

What if, instead of *engineering* shortcuts into the network, the network *learned* its own shortcuts based on which paths actually work?

The strange-but-perfect analogy: the human brain.

The Engineering Problem That Brains Already Solved

A single neuron in your brain connects to roughly 10,000 others through structures called **synapses**. There are about 86 billion neurons total. Any given neuron *could* connect to almost any other, but maintaining synapses is energetically expensive — so neurons are stuck with a hard cap, surrounded by far more potential partners than they can afford to keep.

How does the brain decide *which* connections to keep?

This is the same problem facing a peer in a peer-to-peer network. A web browser can only maintain about 50–100 simultaneous WebRTC connections (the technology that lets browsers talk directly to each other). The network might have hundreds of thousands of nodes. Which 50 do you keep?

The brain solved this problem hundreds of millions of years ago. The solution is called **long-term potentiation**, or LTP.

The Brain's Trick: "Neurons That Fire Together, Wire Together"

In 1973, two scientists — Bliss and Lømo — ran a now-classic experiment. They zapped a particular pathway in a rabbit's brain with high-frequency electrical pulses. Afterward, that pathway responded *more strongly* to subsequent signals — not for seconds, but for *weeks*. The connections had physically gotten stronger.

Decades of follow-up research filled in the details. Here's the simplified version:

1. **The coincidence detector.** Synapses have a special kind of receptor (called NMDA) that only activates when *both* sides of the connection fire at roughly the same time. It's not "I fired" or "you fired" — it's specifically "we fired together."
2. **The strengthening signal.** When that coincidence happens, calcium rushes in and triggers the synapse to insert *more* of a different kind of receptor (AMPA), making the connection physically more sensitive. Next time, the same input gets a stronger response.
3. **Consolidation.** Over the next half hour or so, new proteins get made and new physical structures grow. The change becomes permanent.
4. **Decay.** Connections that *don't* get used regularly weaken over time, in a process called long-term depression (LTD).
5. **A protective tag.** Crucially, recently strengthened synapses get a temporary "tag" that protects them from being overwritten by competing changes — but only for a brief window. Without this tag, learning would be self-destructive: the synapses you just learned were useful would be the *first* ones overwritten by the next signal.

Donald Hebb summarized this in 1949 — before the molecular details were known — in the rule that's now bedrock in neuroscience and AI:

> ***Neurons that fire together, wire together.***

Every artificial neural network you've ever heard of — every part of ChatGPT, every image recognition system, every recommendation algorithm — ultimately runs on a digital descendant of this rule.

Translating Neurons into Network Routing

The brain's mechanism translates *literally* to peer-to-peer routing — no metaphor needed.

NH-1 is the current state-of-the-art N-DHT implementation. It is the result of a long sequence of experiments (NX-1 through NX-17) collapsing into twelve rules and twelve parameters governed by a single biology-derived equation. When this document refers to a specific number — a parameter value, a measured benchmark, a rule count — it is a number from NH-1. When it refers to the family of neuromorphic routing protocols generically, it says **N-DHT**.

The mapping from neurons to NH-1's routing logic:

In the brain...	In NH-1...
A synapse (neural connection)	A connection to a peer
Synaptic strength (weight 0 to 1)	A learned weight on each connection
LTP: co-firing strengthens	A successful lookup increments the weights of all connections it used
LTD: disuse weakens	Every connection slowly decays over time
Synaptic tagging (protection window)	Recently used connections are protected for a window of time
Pruning unused connections	The lowest-scoring connection gets evicted when a new one wants in

That is what NH-1 *is*: a routing system where every connection has a weight that goes up when used and decays when not. The network's "memory" is its routing table, and the routing table evolves into whatever shape best serves the actual traffic.

The Vitality Function

The core of NH-1 is a single equation that scores every connection:

> ***vitality = weight × recency***

The **weight** is a number between 0 and 1, updated like this:

- Every successful lookup that uses the connection raises its weight by 0.05 (capped at 1).
- Every "tick" of the clock multiplies every weight by 0.995 (slow decay).

The **recency** is 1.0 for the first 20 ticks after a connection is used — the protective tag from synaptic tagging — then drops off exponentially.

When a new connection wants in but the routing table is full, the connection with the lowest vitality gets evicted. Frequently used connections accumulate weight and stay. Unused ones decay and get evicted. Recently strengthened ones are temporarily protected.

That's it. That's the core.

Why the Consolidation Matters

The previous version of this system (called NX-17) had **18 different rules and 44 parameters** controlling things like "how full should each part of the routing table be?" and "when should we evict a peer?" and "how much should we prefer nearby peers?"

NH-1 replaces all of that with the single vitality function plus **12 rules and 12 parameters**. Letting the connections themselves learn through reinforcement collapses a tangle of rules into one principle.

The Five Operations

Every behavior in N-DHT falls into one of five categories — and the categories mirror how *any* adaptive system works:

1. NAVIGATE — pick the next hop for a lookup. N-DHT scores each candidate by combining XOR distance progress, learned weight, and observed latency. The latency factor halves the score every 100 ms — so a peer that's mathematically a *bit* further but physically a *lot* closer wins.

2. LEARN — strengthen what works. Four learning mechanisms run on every successful lookup:

- **LTP**: every connection on a *fast* successful path (one that beats the running average of recent path latencies) gets stronger.
- **Hop caching**: every intermediate node along the path remembers the *destination*, not just the next hop. So next time, the path is shorter.
- **Triadic closure**: if you keep seeing peer A relay messages to peer C through you, you introduce them directly. The triangle "A–you–C" becomes a direct edge "A–C." (This is named after social network theory: dense triangles emerge in any network shaped by interaction.)
- **Incoming promotion**: peers who keep reaching out to you become candidates for *your* routing table. Passive learning — the network notices who's interested in you, not just who you're interested in.

3. FORGET — decay everything that isn't reinforced; evict by vitality.

4. EXPLORE — inject occasional randomness. A purely greedy router would lock onto the first decent shortcut and never find better ones. N-DHT has two exploration tricks: a "temperature" that decays over time but spikes when something breaks (heat up when surprised, cool down when stable), and an "epsilon-greedy" rule that picks a random first hop 5% of the time, just to keep options alive.

5. STRUCTURE — bootstrap and maintain the basic shape of the network when new nodes join.

The template is universal: act, learn from feedback, forget the obsolete, occasionally try something new, maintain basic structure. This is how any good adaptive system has to work.

Pub/Sub: Axons, Not Just Synapses

A real peer-to-peer network needs more than just "find the node holding key X." It also needs **broadcast**: a publisher should be able to send a message to all subscribers of a topic, without knowing who they are.

This is publish/subscribe, or "pub/sub." Think of how YouTube notifies subscribers of a new video — except without YouTube being in the middle.

In neuroscience, the *output* of a neuron — the long branching cable that delivers signals to many downstream targets — is called an **axon**. N-DHT builds axonal delivery using a few simple rules:

Topic identity. Every topic has a 64-bit ID, computed offline by anyone who knows the topic. The top 8 bits are the publisher's geographic prefix (the same S2 cell scheme used for node IDs); the bottom 56 bits are a SHA-256 of the topic name. Publisher and subscriber compute the same ID without coordinating — no central registry.

K-closest replication. Instead of routing a publish or subscribe to a single "root" node (which would be a single point of failure), the protocol replicates the topic at the **K nodes in the network whose IDs are XOR-closest to the topic ID** — by default K=5. Five different peers each hold a copy of the subscriber list and a small replay cache. A publisher pushes to those K peers; a subscriber registers at those K peers. As long as any one of them is reachable, the topic works.

Lazy axon promotion. A node that receives a publish but isn't yet hosting the topic *promotes itself* to a role-holder and starts caching messages immediately. When subscribers arrive later, they find the cache already populated. This makes publish-before-subscribe work: a publisher can broadcast even when nobody is listening yet, and the messages wait for up to ~60 seconds in case someone shows up.

Replay on subscribe. When a subscriber attaches to a K-closest axon, the axon replays its cached messages in a single batch — bounded at 100 messages per topic, with a `lastSeenTs` filter so a re-attaching subscriber only gets what it missed, not everything from scratch. Healing and replay are the same mechanism.

Tree self-healing. Subscribers re-issue their subscribe periodically. If an axon died, the re-subscribe naturally lands on whichever K-closest node is now alive — no heartbeats, no failure detection, no parent tracking. Under 5% churn, delivery stays at 100%; after three refresh cycles the tree has fully reformed.

No special "bridge" or relay node. In a sufficiently meshed network, peers communicate directly. A signaling server (used to introduce browsers to each other when the network is bootstrapping) is not in the data path once direct peer connections exist. If the signaling server dies, ongoing pub/sub keeps working through the peer mesh.

In production validation, 100 peers in a single network deliver pub/sub messages with 100% success across multiple topics, including the case where five subscribers join *after* the publishes have already happened and pull them from the axons' replay caches. Each peer plays its own role — pure subscriber, axon role-holder, occasional relay — and the load distributes naturally as the K-closest set varies per topic.

Above this delivery layer sits a feed-style application surface with **five verbs** — `publish`, `subscribe`, `pull`, `reshare`, `metrics`. They cover what a real social or agent-collaboration application asks of a substrate: author new content, attach to a topic, fetch a referenced post on demand, forward with provenance, and let a publisher see verifiable reach without identifying any individual subscriber. Encryption, schema, and ordering belong to the application above this layer; the protocol carries opaque bytes.

The Big Result: Hitting the Theoretical Floor

In 2004, Frank Dabek and colleagues proved a beautiful, depressing result: **no recursive DHT can be faster than 3δ**, where δ is the median one-way latency between random pairs of nodes on the Internet.

The proof is geometric. Each hop in a DHT covers half the remaining distance. So the total time is δ + δ/2 + δ/4 + δ/8 + ... which converges to **2δ**. Add one more hop for final delivery and you get 3δ. No matter how clever your routing, no matter how big your network, you can't beat this.

For two decades, no published DHT had been measured at this floor. The best implementations got to maybe 2× the floor.

NH-1's predecessor, NX-17, hits **1.16× the floor** at 25,000 nodes. NH-1 hits **1.27×**. They sit at the theoretical limit. The remaining ~20% overhead is structural — they take about 4 to 5 hops where an ideal protocol would take 3, and each "extra" hop costs about δ/2, exactly as the geometric series predicts.

For comparison, plain Kademlia *gets worse* as the network grows: 2.01× the floor at 5,000 nodes, 2.65× at 50,000. It scales the wrong way.

Is It Really the Learning, or Just the Geography?

A skeptic would push back: "Sure, but maybe the geographic prefix is doing all the real work. The brain-inspired learning is gravy."

An **ablation study** answers this. (An ablation study removes one feature and re-runs everything to see what that feature actually contributed.) Strip the geographic prefix entirely — random IDs again — and re-run the comparison.

Result: with **zero geographic information**, NX-17 still routes **26% faster than Kademlia**, and NH-1 routes **8% faster**. The learning is doing real work, not sharpening pre-existing geographic structure.

This is the kind of clean control experiment that should accompany any claim of this kind.

The Slice World Test: Healing a Broken Network

The sharpest demonstration is the **Slice World** test. The network gets cut almost in half — Eastern hemisphere on one side, Western on the other, connected only through a *single node* near Hawaii. Every other cross-hemisphere connection is severed.

Can the protocol still route messages between hemispheres?

- **Plain Kademlia: 0% success.** With no learning, the partition is permanent. Messages can't find the bridge.
- **G-DHT (geography only, no learning): 4.6% success.** The geographic prefix accidentally points a few peers at the bridge, but nothing builds on the discovery.
- **NX-17 and NH-1: ~94% success.**

The bridge becomes a **seed crystal**. After just 10 lookups through the partition, hop caching has installed cross-hemisphere edges in many intermediate nodes. Triadic closure creates direct connections between peers that keep meeting through the bridge. By 500 lookups, hundreds of cross-hemisphere connections exist. The partition has effectively dissolved.

The protocol doesn't keep finding the bridge — it *uses* the bridge to rebuild the bridges that were cut. Like a brain forming new pathways around damaged tissue.

How Does This Become Real Software?

The simulator is the lab — fifty thousand simulated peers in a single browser tab, no real network underneath. The same code has to eventually run on the actual internet, where messages take real milliseconds and connections occasionally die. How do you get there?

The trick is keeping two things separate that everyone *wants* to merge: **the protocol** (the rules of routing — AP scoring, hop caching, vitality, axonal trees) and **the network** (the actual machinery that moves bytes between machines). If you tangle them together, the simulator becomes useless the moment you deploy, because the protocol code was wired into a fake network. If you keep them apart, the simulator becomes the deployment vehicle: same protocol, different network underneath.

N-DHT keeps them apart with **two contracts**.

The first contract — call it the **DHT contract** — is what the application above sees. An app like a chat client doesn't care how routing works internally; it just wants to *do things*. So the DHT exposes eight verbs: `start`, `stop`, `join`, `leave` (lifecycle); `lookup`, `subscribe`, `unsubscribe`, `publish` (the actual operations); plus `getMetrics`, `getSynaptome`, and `onEvent` for telemetry — a way for the application to *watch* what the protocol is doing without being able to mess with it.

The second contract — the **Transport contract** — is what the network underneath has to provide. It's deliberately small: open a channel to a peer, close a channel, send a message and wait for the reply, send a message and don't wait, register a callback for when a peer dies, ask for a peer's measured latency. Twelve methods. That's it.

The protocol — the routing logic, the learning rules, all the brain-inspired machinery — sits in between. It calls *down* into the Transport ("send this peer a message asking for its closest synapses to target X") and emits events *up* through the DHT contract ("a lookup just completed; here's the result"). It doesn't know whether the Transport beneath it is the simulator's in-process fake or a real WebRTC connection over the internet. **It can't tell the difference**, by design.

That last point is what matters. When the simulator says "NH-1 takes about 5 hops on average to find a target in a 25,000-node network," that number isn't a simulator artifact. It's a property of the protocol code, which is the same code that will run when this gets deployed. The Transport changes; the protocol doesn't. The simulator's hop counts, latency curves, churn-resilience numbers — they all transfer to the real internet because the routing decisions that produce those numbers are made in code that doesn't know it's being simulated.

The legacy version of N-DHT did *not* have this property. The simulator code was god-like — it could reach into any node's internal state and read it, because they were all in the same process. The first version of the protocol exploited that, because of course it did. Then we spent fifteen commits unbinding the protocol from the god's-eye view: every cross-peer read had to go through the Transport contract, every liveness check had to come from a real heartbeat, every routing decision had to be made by the peer that owns the data, not by the source of the lookup. By the end, the only places the protocol still touches the global node-map are sim-only orchestration — the simulator's equivalent of "spin up a node" and "destroy a node," which production replaces with operating-system-level process startup and shutdown.

The benchmark check at the end of that fifteen-commit pass: 25,000 simulated nodes, before and after. NH-1 came out within five percent on hop counts and one percent on latency — the small drift upward is the architecturally-honest cost of letting each peer make its own decisions instead of having the source orchestrate the walk. The other protocols (Kademlia, G-DHT, NX-17) came out within one percent across the board.

So the simulator is the deployment vehicle, and the plumbing on the other side now exists. A production Transport built on WebRTC data channels — `axona-peer`, the browser-resident node — runs at https://axona.net. A signaling broker — `axona-bridge` — handles the WebRTC offer/answer exchange that two peers behind NATs need to find each other; it runs at https://bridge.axona.net and is interchangeable (any operator can stand one up, and a federated mesh of them is on the roadmap). The cold-start problem — finding your first peer when you've never been on the network before — resolves through any of three `BootstrapEndpoint` variants: a rendezvous URL with a signed manifest, a QR-code-pasted pairing string for direct device-to-device pairing, or an in-process simulator pointer. Once bootstrap returns one open channel, the routing logic is unchanged — the same `lookup_step` chain that the simulator runs.

The product name for this whole stack — the peer, the bridge, the protocol, the SDK that ships them to applications — is **Axona**. The protocol is N-DHT; the network of nodes running it is Axona. The first

application running on it in production is `civildefense.io`, a tap-to-report incident map built in weeks because the substrate primitives (signed posts, geographic locality, 24-hour expiry, anonymous P2P) inherit directly from this protocol layer. Source for the three live components: https://github.com/axona-net/axona-peer, https://github.com/axona-net/axona-bridge, https://github.com/axona-net/dht-sim.

The Honest Footnotes

It is worth being explicit about what *isn't* measured.

The simulator runs about 25,000 lines of JavaScript code modeling the network, but it abstracts away several real-world frictions:

- **Connection setup time.** Real WebRTC connections take 1.5–3 seconds to negotiate. The simulator treats them as instant, so real-world recovery from partitions will be slower than the simulator suggests.
- **Timeout windows.** Real RPCs to dead nodes stall for seconds before failing. The simulator detects death instantly.
- **Bandwidth saturation.** Initially feared as a *success-disaster* failure mode for adaptive routing — that AP scoring's preference for fast peers might funnel traffic onto a few overloaded nodes, oscillating as they collapse and recover. Recent simulations measure the per-node traffic distribution directly and the result is the opposite: at every tested scale (5K–50K nodes), N-DHT distributes load broadly across the population while plain Kademlia and G-DHT concentrate it. At 50,000 nodes, *zero* N-DHT nodes process more than 100× the network mean traffic; K-DHT produces 56 such nodes, G-DHT produces 62. Bandwidth concentration is much less of an issue for N-DHT than for K-DHT — not a non-issue, but not the deploy blocker first feared.
- **Latency jitter.** Real round-trip times vary by ±30% from queuing and congestion. The simulator's latencies are clean and monotone.

These get called out in a separate red-team analysis. The protocol's measured results show **the brain working perfectly**; the *body* — actually deploying it on real internet conditions — still needs work.

What's Next: Plasticity of Plasticity

The most interesting future direction is **metaplasticity** — plasticity of plasticity. In real brains, the rules governing learning *themselves* change based on the brain's activity level. A neuron that's been very active becomes harder to strengthen further (otherwise everything saturates). A neuron that's been quiet becomes easier. The learning rules adapt.

NH-1's parameters — decay rate, protection window, exploration rate — are currently hand-picked constants. A metaplastic version would let the network self-tune them based on local conditions. A peer in a stable region would adapt slowly. A peer in a high-churn region would adapt aggressively. The user would set one knob — "I want my lookup failure rate below 1%" — and the network would tune itself to hit it.

That's the next layer of brain-inspired self-organization, and the natural endpoint of the path the protocol lays out.

Why This Matters

Step back from the technical details. The big idea:

A peer-to-peer network and a brain face the same engineering problem — limited connection capacity, vastly more candidates than slots, and the need to figure out which connections are useful.

The brain's hundreds-of-millions-of-years-old solution maps directly onto network routing. Strengthen what works. Decay what doesn't. Protect recent learning briefly so it isn't immediately overwritten. Occasionally explore. The result is a network whose structure is shaped by the traffic it carries, rather than by rules an engineer guessed at.

This isn't just a faster DHT. It's a demonstration that adaptive systems with the right learning rules find their own structure — that decades of brain research can collapse a 44-parameter system into a 12-parameter one, and that you can hit a theoretical limit that's stood for two decades by importing the right idea from a different field.

Code, data, simulator, and the red-team analysis criticizing it are open source.